ComfyUI on Windows AMD ROCm

DynamicVRAM Stability Notes for 12 GB RDNA2 / gfx1031-class GPUs

Krea2 mixed-precision workloads, WDDM residency, repeated-run slowdown, and a practical recovery workflow

Privacy by design

This edition contains no user name, host name, local directory, motherboard, CPU model, disk layout, monitor topology, device serial, adapter LUID, account information, or other host-specific identifier. Hardware is described only at the GPU architecture / VRAM-class level needed to make the observations technically useful.

Status

Field notes from a reproducible Windows AMD ROCm troubleshooting session. These are observations, not an official ComfyUI or AMD support document. Where the evidence is strong, the guide says so; where causality remains uncertain, it is labeled as a hypothesis.

Public edition v1.0 - October 2026

1. Executive summary

A Windows AMD ROCm ComfyUI setup using a 12 GB RDNA2 / gfx1031-class GPU showed intermittent, extreme sampling slowdowns when running Krea2 mixed-precision models whose staged model size slightly exceeded physical VRAM. Normal runs were roughly in the 16-26 s/it range, while degraded runs reached approximately 35 s/it, 90-100 s/it, and in one extreme case more than 300 s/it.

The strongest telemetry clue was a residency shift: dedicated GPU memory dropped by roughly 2 GB while Shared GPU Memory increased by a similar amount at the same time sampling speed collapsed. This behavior is consistent with a slow system-memory / PCIe path, although the exact fault boundary could be ComfyUI DynamicVRAM, comfy-aimdo, PyTorch/ROCm, WDDM, or an interaction between them.

Several plausible causes were tested and deprioritized: LoRA patching was not required for the slowdown; VRAM stress tests passed; HAGS was unavailable/disabled in the tested driver stack; and a secondary AMD display adapter was not visible to PyTorch and did not contribute compute or pooled VRAM.

The most useful practical change was an in-place upgrade from ComfyUI 0.34.0 / comfy-aimdo 0.4.15 / comfy-kitchen 0.2.31 to ComfyUI 0.42.0 / comfy-aimdo 0.5.6 / comfy-kitchen 0.2.39 while keeping the same PyTorch/ROCm runtime for the first comparison. Repeated Krea2 runs after that update were stable, including heavier four-LoRA, 17-step workflows. A clean Windows restart with Fast Startup disabled and current AMD display drivers was also part of the stable baseline.

Bottom line

If a repeated Krea2 workflow on AMD suddenly becomes many times slower, do not assume the sampler or LoRAs are the cause. Check Dedicated vs Shared GPU Memory, try the /free endpoint before restarting, verify DynamicVRAM/async-offload state, and compare against a clean restart on a current ComfyUI/aimdo stack.

2. Scope and anonymized test environment

The goal of the test was not to produce a universal benchmark. It was to isolate a repeatable failure mode on a Windows ROCm workstation using an over-VRAM Krea2 workflow. Host-specific details have deliberately been removed.

Layer	Public test description
OS	Windows 11; WDDM graphics stack
Primary GPU	AMD Radeon RDNA2, gfx1031-class, 12 GB VRAM
Secondary display GPU	Older AMD Polaris-class adapter; not visible to the active PyTorch/ROCm environment
System memory	Sufficient for CPU offload; exact host configuration intentionally omitted
Model family	Krea2 / FLUX-style mixed-precision workflow
Model staging	~12.86 GB staged for the diffusion model on a ~12.27 GB-reported VRAM device
Text encoder	~4.61 GB staged; CPU offload; fp16 runtime

VAE	~241 MB staged; split-attention path
Typical latent	1 x 4 x 128 x 88 (~1024 x 704 output-class workflow)
Upscale	RRDBNet-class upscaler used in the realistic workflow

The model staging size is intentionally important: the diffusion model alone is slightly larger than reported physical VRAM, so DynamicVRAM/offloading is not optional in the tested configuration.

3. Software stack: before and after

Component	Before	After
ComfyUI	0.34.0	0.42.0
comfy-aimdo	0.4.15	0.5.6
comfy-kitchen	0.2.31	0.2.39
Frontend	1.51.9	1.57.1
PyTorch	2.13.0+rocm10.1.0a20260822	unchanged for the first A/B test
ROCm/HIP as reported by runtime	ROCm (7,16) / HIP 7.16.x	unchanged for the first A/B test

Keeping PyTorch/ROCm unchanged during the first application update was deliberate. It isolated the effect of ComfyUI, aimdo, kitchen, manager-side changes, and the updated AMD-oriented custom nodes from a simultaneous runtime upgrade.

4. Stable launcher baseline used for the post-update tests

The following flags describe the tested state. They are not presented as universally optimal settings; they are simply the configuration under which the stable post-update runs were observed.

```
--cache-none
--disable-smart-memory
--disable-pinned-memory
--enable-manager
--enable-manager-legacy-ui
--disable-triton-backend
--use-quad-cross-attention
```

An older offline/API-disabling flag was also present in the source setup, but it is deprecated in newer ComfyUI and is omitted here because it is unrelated to the memory behavior. Current ComfyUI documentation exposes DynamicVRAM and async-offload flags directly; on supported AMD/ROCm versions DynamicVRAM may be enabled automatically.

5. Failure signature

- Normal Krea2 sampling: roughly 16-26 s/it depending on run, sampler, and initialization state.
- Mild degradation: ~35 s/it.
- Severe degradation: ~90-100 s/it.
- Extreme observed state: >300 s/it, including a run with no LoRA patches attached.

- During severe slowdowns, the main GPU could become quieter rather than louder, suggesting data starvation/waiting rather than full compute saturation.
- A full Windows restart could restore normal speed when restarting only ComfyUI did not always do so.

6. WDDM residency telemetry: the strongest clue

Metric	Healthy snapshot	Severely degraded snapshot
Dedicated GPU memory	~10.47 GB	~8.48 GB
Shared GPU memory	~0.27 GB	~2.51 GB
Per-process RAM	~4.36 GB	~5.22 GB
Copy-engine activity	present	near zero in the captured bad state

The raw WDDM compute-engine sum could exceed 100% because multiple engines were added together; that absolute percentage should not be interpreted as ordinary GPU utilization. The useful signal was the memory-residency shift: approximately 2 GB left Dedicated GPU Memory while a similar amount appeared in Shared GPU Memory exactly when performance collapsed.

> **Working mechanism (hypothesis, not proven root cause)**
>
> An over-VRAM model is normally streamed/offloaded through DynamicVRAM. In a bad runtime state, residency appears to move a substantial working set into Shared GPU Memory, making the sampling loop far more dependent on system RAM / PCIe transfers. The precise layer responsible was not conclusively identified.

7. What the tests ruled out or deprioritized

LoRA patching was not the root cause.

A severely slow run occurred with 0 patches attached, so LoRA processing is not a necessary condition. Additional LoRAs may still change memory pressure and the threshold at which a bug appears.

Physical VRAM failure became unlikely.

Both installed AMD adapters completed the standard memtest_vulkan test without reported errors. This does not measure “remaining VRAM life” the way SSD SMART does, but it is useful for detecting data-integrity instability under load.

HAGS was not an active variable.

The tested AMD drivers reported Hardware Scheduling as AlwaysOff / disabled, so Hardware Accelerated GPU Scheduling could not explain the runtime transition.

The secondary GPU did not pool VRAM with the primary GPU.

PyTorch reported a single visible ROCm device. The older secondary adapter therefore did not act as a second ComfyUI compute device and could not add its VRAM to the main model budget.

8. Windows Fast Startup and clean restart

Fast Startup was enabled during part of the investigation. Because Windows Fast Startup can preserve kernel/driver state across a normal shutdown/startup cycle, it was disabled before the later comparison tests. A Restart was used to force a fresh graphics-driver session.

```
# Read Fast Startup state
Get-ItemProperty `
'HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Power' `
-Name HiberbootEnabled |
Select-Object HiberbootEnabled

# Disable Fast Startup (Administrator PowerShell)
Set-ItemProperty `
'HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Power' `
-Name HiberbootEnabled -Value 0
```

This did not prove Fast Startup was the underlying bug. It removed one source of state persistence and made reboots more deterministic for testing.

9. Driver hygiene

The AMD display stack was updated before the final validation. Exact product SKUs are intentionally omitted; the useful public information is that the RDNA2 adapter moved to a current 2026 driver branch and the older Polaris-class adapter was moved off a much older 2023 Windows Update driver onto its supported 2026 branch.

Adapter class	Driver version used in final tests
12 GB RDNA2 / gfx1031-class	32.0.21045.16010
Older Polaris-class display adapter	31.0.21925.1001

```
Get-CimInstance Win32_PnPSignedDriver |
Where-Object { $_.DeviceName -match 'Radeon' } |
Select-Object DeviceName,DriverVersion,DriverDate |
Format-Table -AutoSize
```

10. Application update that materially changed the result

The AMD-specific ComfyUI distribution was updated with its own updater rather than a manual git pull. The application updater changed ComfyUI, comfy-aimdo, comfy-kitchen, the frontend, ComfyUI-Manager, and AMD-oriented custom nodes, while the ROCm/PyTorch updater was deliberately not run for the first comparison.

```
# Run from the ComfyUI ROCm installation root
.\comfyui-rocm-updater.bat

# Keep the separate ROCm/PyTorch updater for a later isolated test
# .\rocm-pytorch-package-updater.bat
```

After the update, DynamicVRAM remained enabled and async weight offloading still used two streams. The major behavioral difference was that repeated heavy Krea2 runs remained stable instead of falling into the previously observed catastrophic slowdown state.

11. Performance observations

These timings are field measurements, not standardized benchmarks. They are useful mainly for the shape of the failure and the before/after stability difference.

Pre-update observation	Result
No LoRA	~22.29 s/it; ~361 s total
LoRA-active run	~20.73 s/it; ~334 s total
Warm fast runs	~17.34-17.43 s/it; ~275-288 s total
Same session, degraded	~35.28 s/it; ~550 s total
Extreme bad state, no LoRA patches	~331 s/it early in the sampling loop
cache-none initial runs	~24-26 s/it, stable but slower than warm LRU runs

Post-update workload	Observed result
2 LoRAs + upscale, 15 steps	~337 s total; sampling settled near ~19 s/it
3 LoRAs + upscale, 15 steps	~285 s total; ~16.7 s/it observed
Another 15-step run + upscale	17.34 s/it; ~279 s total
4 LoRAs, harder prompt, DPM++ 2M SDE, 17 steps	stable ~17.2 s/it after initialization; ~337 s total
Identical repeat of the heavy DPM++ run	~345 s total; only ~2.5% slower overall
Same heavy workflow, Euler ancestral, 17 steps	~311 s total; stable; ~7.8% faster than the first DPM++ run

The key post-update observation was not the absolute benchmark. It was the absence, across repeated runs, of the old 20 -> 90 -> 300 s/it-style collapse.

12. Sampler and step-count notes

In this Krea2 + LoRA workload, Euler ancestral was both faster and subjectively preferable to DPM++ 2M SDE. That is not a universal quality claim; sampler preference is model- and workflow-dependent. It is a reminder that a “more sophisticated” sampler is not automatically better for a given fine-tune/LoRA combination.

A useful health check is to reduce step count while leaving the rest of the workflow unchanged. In a healthy state, seconds per iteration should remain broadly similar while total sampling time falls roughly with the number of steps. If total time stops scaling with work or s/it rises sharply as the session ages, suspect runtime state rather than prompt complexity alone.

13. High-value diagnostic: /free before rebooting

When the slowdown returns, do not immediately reboot. First test whether clearing ComfyUI model/memory state restores performance. This helps localize the failure layer.

```
Invoke-RestMethod `
  -Method Post `
  -Uri "http://127.0.0.1:8188/free" `
  -ContentType "application/json" `
  -Body '{"unload_models":true,"free_memory":true}'
```

Result after /free	Interpretation
Speed immediately returns	Strongly implicates ComfyUI model-manager / DynamicVRAM residency state
No change, but restarting ComfyUI helps	More likely process-local AIMDO / HIP / allocator state
Restarting ComfyUI does not help, but Windows restart does	More likely driver / WDDM / HIP virtual-memory state

This diagnostic is also consistent with a separate public ComfyUI issue in which repeated AMD ROCm Krea2 runs slow dramatically and POST /free restores the fast state.

14. Testing VRAM integrity

Consumer Radeon cards do not expose an SSD-like “wear percentage” for VRAM. The useful practical question is whether the memory returns incorrect data or becomes unstable under sustained access. memtest_vulkan is a simple cross-vendor option that can test each Vulkan-visible GPU independently.

```
# Example: run the downloaded standalone executable
.\memtest_vulkan-v0.5.0.exe


# Verify the selected device in the program output.
# Let the standard 5-minute test complete; any reported data errors matter.
```

Passing this test does not prove every possible hardware fault is absent, but in this investigation it made a physical VRAM defect much less likely than a residency/offload problem.

15. Mixed-generation multi-GPU notes

A second, older AMD display adapter can still be useful for desktop duties or software that supports it through another backend. However, if the active PyTorch/ROCm build reports only one device, the secondary card cannot be used as gpu:1 for ComfyUI nodes and its VRAM is not pooled with the primary card.

```
<COMFY_ROOT>\python_env\python.exe -c "import torch; \
print(torch.__version__); print(torch.version.hip); \
print(torch.cuda.device_count()); \
[print(i, torch.cuda.get_device_name(i)) for i in range(torch.cuda.device_count())]"
```

Mixed-driver, mixed-generation WDDM behavior can still change the desktop/display environment, so it is a reasonable A/B variable if unexplained instability persists: move displays to the primary adapter, disable the secondary adapter temporarily, reboot, and rerun the same workflow. In the documented test series, this remained a possible trigger rather than a proven root cause.

16. Practical troubleshooting order

- Record the failing run before changing anything: s/it, Dedicated GPU Memory, Shared GPU Memory, GPU clocks/fans, and whether the model reports 0 or non-zero patches.
- Call /free and repeat the exact workflow.
- If still slow, restart ComfyUI and repeat.
- If still slow, perform a true Windows Restart (not just shutdown/startup when Fast Startup is enabled).
- Verify AMD display-driver versions and HAGS state with dxdiag.
- Run a VRAM integrity test at stock memory clocks/timings.
- Update ComfyUI/aimdo using the distribution-specific updater; keep the ROCm/PyTorch updater separate at first so the changed layer is identifiable.
- Only after the above, isolate async offload, cache policy, custom nodes, and finally the ROCm/PyTorch package version one variable at a time.

17. Public reproducibility checklist

Check	Expected / useful state
GPU architecture	RDNA2 gfx1031-class visible to PyTorch/ROCm
Model pressure	Diffusion model staged size at or slightly above physical VRAM
DynamicVRAM	Enabled
Async offload	Enabled; 2 streams in the tested baseline
Cache policy	cache-none for the conservative stability test
GPU tuning	Stock/default for diagnosis
Fast Startup	Disabled during reproducibility testing
VRAM test	Pass at stock clocks/timings
Repeated-run test	At least 3-5 identical generations in one ComfyUI session
Failure discriminator	/free test before reboot

18. Warnings that were not sufficient by themselves to explain the slowdown

- Missing SoX for unrelated audio functionality.
- Ollama not reachable for an unrelated prompt-conversion node.
- Legacy frontend API deprecation warnings from custom extensions.
- MIOpen / CK grouped-convolution warning on gfx1031 observed around the VAE stage.
- Brownian interval rounding warning near the end of some sampler runs.

Warnings should still be investigated if they affect the user's workflow, but none of the above correlated with the large sampler-speed collapse strongly enough to be treated as the primary cause in this test series.

19. Public references

patientx-cfz/comfyui-rocm - https://github.com/patientx-cfz/comfyui-rocm
Windows-focused ComfyUI ROCm distribution; documents the app updater and separate ROCm/PyTorch updater.

ComfyUI startup flags - https://github.com/Comfy-Org/docs/blob/main/development/comfyui-server/startup-flags.mdx
Documents DynamicVRAM, async offload, smart-memory and pinned-memory flags.

ComfyUI issue #16502 - https://github.com/Comfy-Org/ComfyUI/issues/16502
AMD ROCm / DynamicVRAM repeated-run slowdown where /free restores performance; highly relevant comparison.

ComfyUI issue #16878 - https://github.com/Comfy-Org/ComfyUI/issues/16878
gfx1031 / 12 GB-class AMD report involving DynamicVRAM instability.

memtest_vulkan - https://github.com/GpuZelenograd/memtest_vulkan
Standalone Vulkan VRAM stress/integrity test.

20. Final conclusion

The strongest lesson from this field investigation is methodological: on Windows AMD ROCm, an over-VRAM ComfyUI workload can fail as a runtime-state problem rather than as a simple “not enough VRAM” error. Large jumps in Shared GPU Memory, quieting GPU fans, and multi-fold increases in s/it are more informative than the mere presence of LoRAs or a complex prompt.

For the documented RDNA2/gfx1031-class workload, updating ComfyUI/aimdo while preserving the same PyTorch/ROCm runtime, using a clean Windows restart, current display drivers, stock GPU tuning, and a conservative cache policy produced repeated stable runs that scaled normally with sampler/workload changes. This does not prove a universal fix, but it provides a practical diagnostic path and a reproducible baseline for similar AMD systems.

Shareable summary

If you reproduce this class of slowdown, capture Dedicated vs Shared GPU Memory, test /free before rebooting, report the exact ComfyUI + aimdo + PyTorch/ROCm versions, and note whether the problem occurs with 0 patches attached. Those details are far more useful to developers than a generic “AMD is slow” report.